



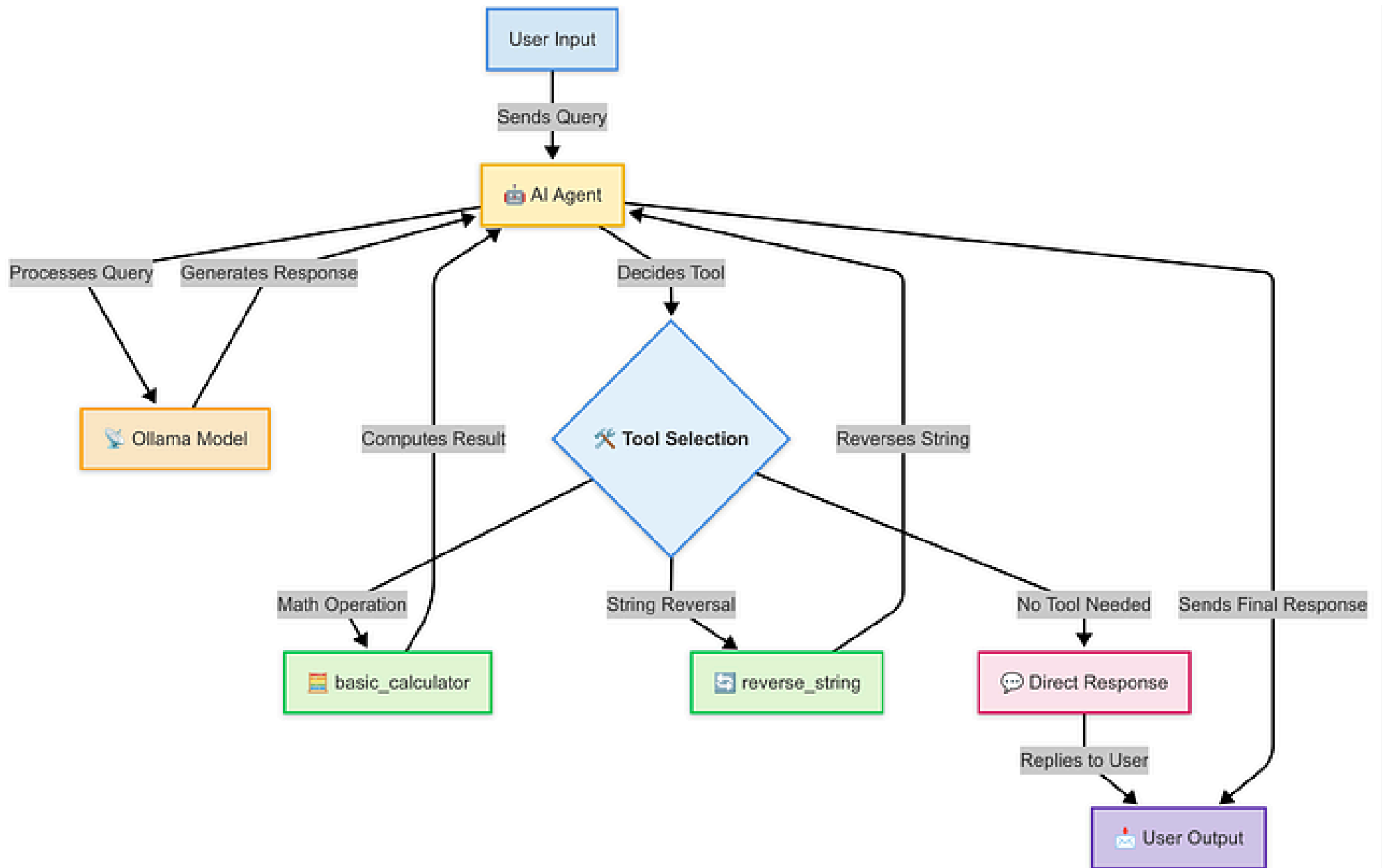


Image by Author

AI Agents: Build an Agent from Scratch (Part-2)

Discover AI agents, their design, and real-world applications.



Vipra Singh

9 min read

[Download article](#) 

Contents 10

- 1 Posts in this Series
- 2 Table of Contents
- 3 1. What is an Agent?
- 4 2. Implementation
 - 2.1 Pre-requisites

[Show all 10 sections](#)

Posts in this Series

1. [Introduction](#) 

2. ***Build an Agent from Scratch (This Post)***

- 3. [AI Agent Frameworks](#) ↗
- 4. [Types of AI Agents](#) ↗
- 5. [Workflow vs Agent](#) ↗
- 6. [Agent Architectures](#) ↗
- 7. [Multi-Agent Architectures](#) ↗
- 8. [Building Multi-Agent System](#) ↗
- 9. [Agentic Memory](#) ↗
- 10. [Agentic RAG](#) ↗
- 11. [Model Context Protocol \(MCP\)](#) ↗
- 12. **Evaluation**

Table of Contents

· [1. What is an Agent?](#) · [2. Implementation](#) ◦ [2.1 Pre-requisites](#) ◦ [2.2 Implementation Steps](#) · [3. Conclusion](#)

In our previous blog post, we provided a comprehensive overview of AI agents, discussing their characteristics, components, evolution, challenges, and future possibilities.

In this blog, we'll explore how to build an agent from scratch using Python. This agent will be capable of making decisions based on user input, selecting appropriate tools, and executing tasks accordingly. Let's get started!

1. What is an Agent?

An agent is an autonomous entity capable of perceiving its environment, making decisions, and taking actions to achieve specific goals. Agents can vary in complexity from simple reactive agents that respond to stimuli to more advanced intelligent agents that learn and adapt over time. Common types of agents include:

1. **Reactive Agents:** Respond directly to environmental changes without internal memory.
2. **Model-Based Agents:** Use internal models of the world to make decisions.
3. **Goal-Based Agents:** Plan actions based on achieving specific goals.
4. **Utility-Based Agents:** Evaluate potential actions based on a utility function to maximize outcomes.

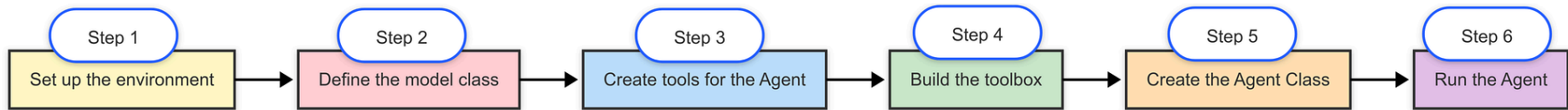
Examples include chatbots, recommendation systems, and autonomous vehicles, each utilizing different types of agents to perform tasks efficiently and intelligently.

The core components of our agent are:

- **Models:** The brain of the agent, responsible for processing the input and generating responses.
- **Tools:** Pre-defined functions that the agent can execute based on the user's request.
- **Toolbox:** A collection of tools the agent has at its disposal.
- **System Prompt:** The instruction set that guides the agent on how to handle user input and choose the right tools.

2. Implementation

Now, let's roll up our sleeves and start building!



2.1 Pre-requisites

The complete code for this tutorial is available in the [AI Agents GitHub repository](#).

You can find the implementation here: [Build an Agent from Scratch](#) [↗](#).

Before running the code, ensure that your system meets the following prerequisites:

1. Python Environment Setup

You'll need Python installed to run the AI agent. Follow these steps to set up your environment:

Install Python (if not already installed)

- Download and install Python (3.8+ recommended) from [python.org](#) [↗](#).
- Verify installation:

```
1 python --version
```

Create a Virtual Environment (Recommended) It's best to use a virtual environment to manage dependencies:

```
1 python -m venv ai_agents_env
2 source ai_agents_env/bin/activate # On Windows: ai_agents_env\Scripts\activate
```

Install Required Dependencies Navigate to the repository directory and install dependencies:

```
1 pip install -r requirements.txt
```

2. Setup Ollama Locally

Ollama is used to run and manage local language models efficiently. Follow these steps to install and configure it:

Download and Install Ollama

- Visit [Ollama's official site](#) and download the installer for your OS.
- Install it following the instructions for your platform.

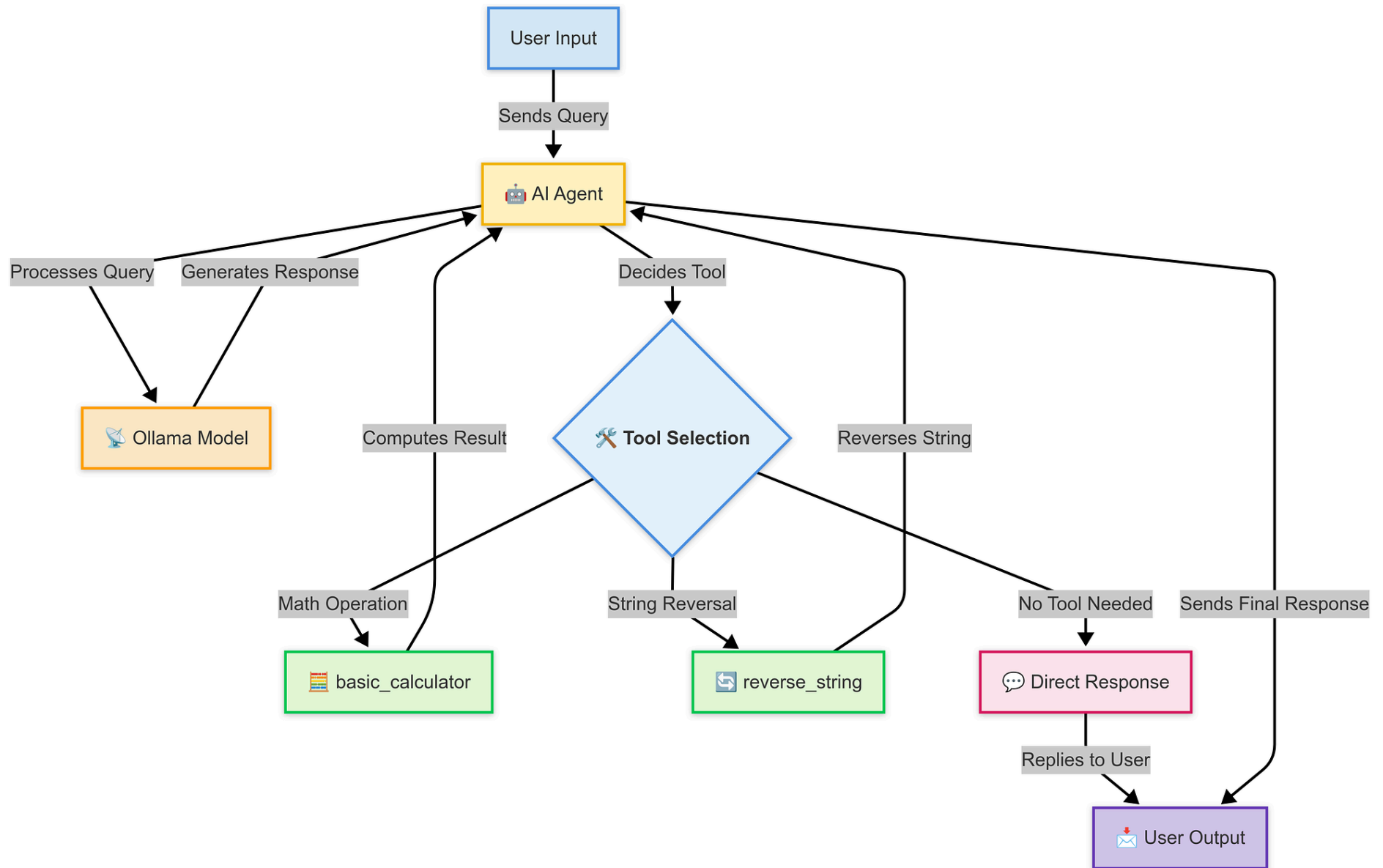
Verify Ollama Installation Run the following command to check if Ollama is installed correctly:

```
1 ollama --version
```

Pull a Model (if required) Some agent implementations may require a specific model. You can pull a model using:

```
1 ollama pull mistral # Replace 'mistral' with the model needed
```

2.2 Implementation Steps



Step 1: Setting Up the Environment

Along with Python, we'll also need to install some essential libraries. For this tutorial, we'll be using `requests` , `json` , and `termcolor` . Additionally, we will use `dotenv` to manage environment variables.

```
1 pip install requests termcolor python-dotenv
```

Step 2: Defining the Model Class

The first thing we need is a model that will process user input. We'll create an `OllamaModel` class, which interacts with a local API to generate responses.

Here's a basic implementation:

```
1 from termcolor import colored
2 import os
3 from dotenv import load_dotenv
4 load_dotenv()
5 ### Models
6 import requests
7 import json
8 import operator
9 class OllamaModel:
10     def __init__(self, model, system_prompt, temperature=0, stop=None):
11         """
12         Initializes the OllamaModel with the given parameters.
13
14         Parameters:
15         model (str): The name of the model to use.
16         system_prompt (str): The system prompt to use.
```

```
17     temperature (float): The temperature setting for the model.
18     stop (str): The stop token for the model.
19     """
20     self.model_endpoint = "http://localhost:11434/api/generate"
21     self.temperature = temperature
22     self.model = model
23     self.system_prompt = system_prompt
24     self.headers = {"Content-Type": "application/json"}
25     self.stop = stop
26
27     def generate_text(self, prompt):
28         """
29         Generates a response from the Ollama model based on the provided prompt.
30
31         Parameters:
32         prompt (str): The user query to generate a response for.
33
34         Returns:
35         dict: The response from the model as a dictionary.
36         """
37         payload = {
38             "model": self.model,
39             "format": "json",
40             "prompt": prompt,
41             "system": self.system_prompt,
42             "stream": False,
43             "temperature": self.temperature,
44             "stop": self.stop
45         }
46
47         try:
```

```

48         request_response = requests.post(
49             self.model_endpoint,
50             headers=self.headers,
51             data=json.dumps(payload)
52         )
53
54         print("REQUEST RESPONSE", request_response)
55         request_response_json = request_response.json()
56         response = request_response_json['response']
57         response_dict = json.loads(response)
58
59         print(f"\n\nResponse from Ollama model: {response_dict}")
60
61         return response_dict
62     except requests.RequestException as e:
63         response = {"error": f"Error in invoking model! {str(e)}"}
64         return response

```

This class initializes with the model's name, system prompt, temperature, and stop token. The `generate_text` method sends a request to the model API and returns the response.

Step 3: Creating Tools for the Agent

The next step is to create tools that our agents can use. These tools are simple Python functions that perform specific tasks. Here's an example of a basic calculator and a string reverser:

```

1 def basic_calculator(input_str):
2     """
3     Perform a numeric operation on two numbers based on the input string or dictionary.

```

Parameters:

`input_str` (str or dict): Either a JSON string representing a dictionary with keys 'num1', 'num2', and 'operation' or a dictionary directly. Example: '{"num1": 5, "num2": 3, "operation": "add"}' or {"num1": 67869, "num2": 9030393, "operation": "divide"}

Returns:

str: The formatted result of the operation.

Raises:

Exception: If an error occurs during the operation (e.g., division by zero).

ValueError: If an unsupported operation is requested or input is invalid.

"""

try:

 # Handle both dictionary and string inputs

 if isinstance(input_str, dict):

 input_dict = input_str

 else:

 # Clean and parse the input string

 input_str_clean = input_str.replace("'", "\\'")

 input_str_clean = input_str_clean.strip().strip("\\'")

 input_dict = json.loads(input_str_clean)

 # Validate required fields

 if not all(key in input_dict for key in ['num1', 'num2', 'operation']):

 return "Error: Input must contain 'num1', 'num2', and 'operation'"

 num1 = float(input_dict['num1']) # Convert to float to handle decimal numbers

 num2 = float(input_dict['num2'])

 operation = input_dict['operation'].lower() # Make case-insensitive

except (json.JSONDecodeError, KeyError) as e:

```
35         return "Invalid input format. Please provide valid numbers and operation."
36     except ValueError as e:
37         return "Error: Please provide valid numerical values."
38
39     # Define the supported operations with error handling
40     operations = {
41         'add': operator.add,
42         'plus': operator.add, # Alternative word for add
43         'subtract': operator.sub,
44         'minus': operator.sub, # Alternative word for subtract
45         'multiply': operator.mul,
46         'times': operator.mul, # Alternative word for multiply
47         'divide': operator.truediv,
48         'floor_divide': operator.floordiv,
49         'modulus': operator.mod,
50         'power': operator.pow,
51         'lt': operator.lt,
52         'le': operator.le,
53         'eq': operator.eq,
54         'ne': operator.ne,
55         'ge': operator.ge,
56         'gt': operator.gt
57     }
58
59     # Check if the operation is supported
60     if operation not in operations:
61         return f"Unsupported operation: '{operation}'. Supported operations are: {' '.join(operations.keys())}"
62
63     try:
64         # Special handling for division by zero
65         if (operation in ['divide', 'floor_divide', 'modulus']) and num2 == 0:
```

```
66         return "Error: Division by zero is not allowed"
67
68     # Perform the operation
69     result = operations[operation](num1, num2)
70
71     # Format result based on type
72     if isinstance(result, bool):
73         result_str = "True" if result else "False"
74     elif isinstance(result, float):
75         # Handle floating point precision
76         result_str = f"{result:.6f}".rstrip('0').rstrip('.')
77     else:
78         result_str = str(result)
79
80     return f"The answer is: {result_str}"
81 except Exception as e:
82     return f"Error during calculation: {str(e)}"
83
84 def reverse_string(input_string):
85     """
86     Reverse the given string.
87
88     Parameters:
89     input_string (str): The string to be reversed.
90
91     Returns:
92     str: The reversed string.
93     """
94     # Check if input is a string
95     if not isinstance(input_string, str):
96         return "Error: Input must be a string"
```

```

97
98     # Reverse the string using slicing
99     reversed_string = input_string[::-1]
100
101     # Format the output
102     result = f"The reversed string is: {reversed_string}"
103
104     return result

```

These functions are designed to perform specific tasks based on the input provided. The `basic_calculator` handles arithmetic operations, while `reverse_string` reverses a given string.

Step 4: Building the Toolbox

The `ToolBox` class stores all the tools the agent can use and provides descriptions for each:

```

1  class Toolbox:
2      def __init__(self):
3          self.tools_dict = {}
4
5      def store(self, functions_list):
6          """
7          Stores the literal name and docstring of each function in the list.
8
9          Parameters:
10         functions_list (list): List of function objects to store.
11
12         Returns:

```

```

13         dict: Dictionary with function names as keys and their docstrings as values.
14         """
15         for func in functions_list:
16             self.tools_dict[func.__name__] = func.__doc__
17         return self.tools_dict
18
19     def tools(self):
20         """
21         Returns the dictionary created in store as a text string.
22
23         Returns:
24         str: Dictionary of stored functions and their docstrings as a text string.
25         """
26         tools_str = ""
27         for name, doc in self.tools_dict.items():
28             tools_str += f"{name}: \"{doc}\"\\n"
29         return tools_str.strip()

```

This class will help the agent understand which tools are available and what each one does.

Step 5: Creating the Agent Class

The agent needs to think, decide which tool to use, and execute it. Here's the `Agent` class:

```

1 agent_system_prompt_template = """
2 You are an intelligent AI assistant with access to specific tools. Your responses must ALWAYS be in this JS
3 {{
4     "tool_choice": "name_of_the_tool",
5     "tool_input": "inputs_to_the_tool"

```



```
6  }}
7
8  TOOLS AND WHEN TO USE THEM:
9
10 1. basic_calculator: Use for ANY mathematical calculations
11   - Input format: {"num1": number, "num2": number, "operation": "add/subtract/multiply/divide"}}
12   - Supported operations: add/plus, subtract/minus, multiply/times, divide
13   - Example inputs and outputs:
14     Input: "Calculate 15 plus 7"
15     Output: {"tool_choice": "basic_calculator", "tool_input": {"num1": 15, "num2": 7, "operation": "add'
16
17     Input: "What is 100 divided by 5?"
18     Output: {"tool_choice": "basic_calculator", "tool_input": {"num1": 100, "num2": 5, "operation": "div
19
20 2. reverse_string: Use for ANY request involving reversing text
21   - Input format: Just the text to be reversed as a string
22   - ALWAYS use this tool when user mentions "reverse", "backwards", or asks to reverse text
23   - Example inputs and outputs:
24     Input: "Reverse of 'Howwww'?"
25     Output: {"tool_choice": "reverse_string", "tool_input": "Howwww"}}
26
27     Input: "What is the reverse of Python?"
28     Output: {"tool_choice": "reverse_string", "tool_input": "Python"}}
29
30 3. no tool: Use for general conversation and questions
31   - Example inputs and outputs:
32     Input: "Who are you?"
33     Output: {"tool_choice": "no tool", "tool_input": "I am an AI assistant that can help you with calcula
34
35     Input: "How are you?"
36     Output: {"tool_choice": "no tool", "tool_input": "I'm functioning well, thank you for asking! I'm her
```

```
37
38 STRICT RULES:
39 1. For questions about identity, capabilities, or feelings:
40     - ALWAYS use "no tool"
41     - Provide a complete, friendly response
42     - Mention your capabilities
43
44 2. For ANY text reversal request:
45     - ALWAYS use "reverse_string"
46     - Extract ONLY the text to be reversed
47     - Remove quotes, "reverse of", and other extra text
48
49 3. For ANY math operations:
50     - ALWAYS use "basic_calculator"
51     - Extract the numbers and operation
52     - Convert text numbers to digits
53
54 Here is a list of your tools along with their descriptions:
55 {tool_descriptions}
56
57 Remember: Your response must ALWAYS be valid JSON with "tool_choice" and "tool_input" fields.
58 """
59 class Agent:
60     def __init__(self, tools, model_service, model_name, stop=None):
61         """
62         Initializes the agent with a list of tools and a model.
63
64         Parameters:
65         tools (list): List of tool functions.
66         model_service (class): The model service class with a generate_text method.
67         model_name (str): The name of the model to use.
```

```
68         """
69         self.tools = tools
70         self.model_service = model_service
71         self.model_name = model_name
72         self.stop = stop
73
74     def prepare_tools(self):
75         """
76         Stores the tools in the toolbox and returns their descriptions.
77
78         Returns:
79         str: Descriptions of the tools stored in the toolbox.
80         """
81         toolbox = ToolBox()
82         toolbox.store(self.tools)
83         tool_descriptions = toolbox.tools()
84         return tool_descriptions
85
86     def think(self, prompt):
87         """
88         Runs the generate_text method on the model using the system prompt template and tool descriptions.
89
90         Parameters:
91         prompt (str): The user query to generate a response for.
92
93         Returns:
94         dict: The response from the model as a dictionary.
95         """
96         tool_descriptions = self.prepare_tools()
97         agent_system_prompt = agent_system_prompt_template.format(tool_descriptions=tool_descriptions)
98
```

```
99         # Create an instance of the model service with the system prompt
100
101     if self.model_service == OllamaModel:
102         model_instance = self.model_service(
103             model=self.model_name,
104             system_prompt=agent_system_prompt,
105             temperature=0,
106             stop=self.stop
107         )
108     else:
109         model_instance = self.model_service(
110             model=self.model_name,
111             system_prompt=agent_system_prompt,
112             temperature=0
113         )
114
115     # Generate and return the response dictionary
116     agent_response_dict = model_instance.generate_text(prompt)
117     return agent_response_dict
118
119 def work(self, prompt):
120     """
121     Parses the dictionary returned from think and executes the appropriate tool.
122
123     Parameters:
124     prompt (str): The user query to generate a response for.
125
126     Returns:
127     The response from executing the appropriate tool or the tool_input if no matching tool is found.
128     """
129     agent_response_dict = self.think(prompt)
```

```

130     tool_choice = agent_response_dict.get("tool_choice")
131     tool_input = agent_response_dict.get("tool_input")
132
133     for tool in self.tools:
134         if tool.__name__ == tool_choice:
135             response = tool(tool_input)
136             print(colored(response, 'cyan'))
137             return
138
139     print(colored(tool_input, 'cyan'))
140     return

```

This class has three main methods:

- **prepare_tools:** Stores and returns the descriptions of the tools.
- **think:** Decide which tool to use based on the user prompt.
- **work:** Executes the chosen tool and returns the result.

Step 6: Running the Agent

Finally, let's put everything together and run our agent. In your script's `main` section, initialize the agent and start accepting user input:

```

1 # Example usage
2 if __name__ == "__main__":
3     """
4     Instructions for using this agent:

```

```
5
6 Example queries you can try:
7 1. Calculator operations:
8   - "Calculate 15 plus 7"
9   - "What is 100 divided by 5?"
10  - "Multiply 23 and 4"
11
12 2. String reversal:
13   - "Reverse the word 'hello world'"
14   - "Can you reverse 'Python Programming'?"
15
16 3. General questions (will get direct responses):
17   - "Who are you?"
18   - "What can you help me with?"
19
20 Ollama Commands (run these in terminal):
21 - Check available models:      'ollama list'
22 - Check running models:       'ps aux | grep ollama'
23 - List model tags:             'curl http://localhost:11434/api/tags'
24 - Pull a new model:            'ollama pull mistral'
25 - Run model server:           'ollama serve'
26 """"
27
28 tools = [basic_calculator, reverse_string]
29
30 # Uncomment below to run with OpenAI
31 # model_service = OpenAIModel
32 # model_name = 'gpt-3.5-turbo'
33 # stop = None
34
35 # Using Ollama with llama2 model
```

```

36     model_service = OllamaModel
37     model_name = "llama2" # Can be changed to other models like 'mistral', 'codellama', etc.
38     stop = "<eot_id>"
39
40     agent = Agent(tools=tools, model_service=model_service, model_name=model_name, stop=stop)
41
42     print("\nWelcome to the AI Agent! Type 'exit' to quit.")
43     print("You can ask me to:")
44     print("1. Perform calculations (e.g., 'Calculate 15 plus 7')")
45     print("2. Reverse strings (e.g., 'Reverse hello world')")
46     print("3. Answer general questions\n")
47
48     while True:
49         prompt = input("Ask me anything: ")
50         if prompt.lower() == "exit":
51             break
52
53         agent.work(prompt)

```

3. Conclusion

In this blog post, we explored the understanding of what an agent is to implement one step by step. We set up the environment, defined the model, created essential tools, and built a structured toolbox to support our agent's functionality. Finally, we brought everything together by running the agent in action.

This structured approach provides a solid foundation for building intelligent, interactive agents capable of automating tasks and making informed decisions. As AI agents continue to evolve, their applications will expand across industries, driving efficiency and innovation. Stay tuned for more insights and enhancements to take your AI agents to the next level!

Credits

In this blog post, we have compiled information from various sources, including research papers, technical blogs, official documentation, YouTube videos, and more. Each source has been appropriately credited beneath the corresponding images, with source links provided.

Thank you for reading!

If this guide has enhanced your understanding of Python and Machine Learning:

- Please show your support with a clap 🙌 or several claps!
- Your claps help me create more valuable content for our vibrant Python or ML community.
- Feel free to share this guide with fellow Python or AI / ML enthusiasts.
- Your feedback is invaluable — it inspires and guides our future posts.

Connect with me!

[Vipra](#) 



By **Freedium.** – *made with care.*

[About](#)

[Privacy](#)

[Terms](#)

[RSS](#)

[GitHub](#)

[Codeberg](#)

